Review 1

Agentic Shell: Agent 10 – Inode & Symlink Auditor Agent

A natural-language agent for finding broken links, symlink loops and hard links on Linux

Name	Registration No.
Debojyoti Barai Mitul	24BCE2951
Shivam Prakash	24BDS0340

Chapter 1: Introduction

1.1 Project Overview

This project is **Agent 10, the Inode & Symlink Auditor Agent**, one of the 35 agents of our class project, the *Agentic Shell*. In the Agentic Shell, a base shell receives a request written in plain English and routes it to the agent best suited to handle it. Our agent is responsible for everything to do with inodes and links on a Linux system: it finds broken ("dangling") symbolic links and symlink loops, groups files that are hard links of each other, inspects the inode, link count, owner and permissions of any path, reports how many inodes a filesystem has left, flags suspicious links, and, only after the user confirms, deletes dead links or creates and repoints links. A typical request is *"Find all broken shortcuts on my desktop and delete them."*

Technically, the agent exposes Linux capabilities as **tools for LLM function calling**. A large language model (LLM) reads the request, decides which tools to run and with which arguments, and our Python code performs the real work with system calls such as `lstat()`, `readlink()`, `statvfs()` and `unlink()`. The agent runs on its own today (a chat mode, one-shot requests and direct commands that need no LLM) and offers a small Python interface so that whatever base shell the class adopts can plug it in.

The evolution of Command Line Interfaces (CLIs)

Command line interfaces are the oldest interactive way of controlling a computer. Early time-sharing systems were driven from teletype terminals, where the user typed a command and the machine printed a reply. Unix turned this into a programmable environment: the Thompson shell of the early 1970s was followed by the Bourne shell (1979), which added scripting constructs, and later by the C shell, the Korn shell and GNU **Bash** (1989), which is still the default interactive shell on most Linux distributions. Microsoft followed a different path with **PowerShell** (2006), which passes structured objects between commands instead of plain text.

Across these generations the CLI kept the same strengths: it is fast, scriptable, precise and works over a remote connection. It also kept the same weakness: the user must already know the exact command, its options and its quoting rules. Modern terminals improved the experience with tab completion, history search and syntax highlighting, but they still expect the user to speak the shell's language.

The shift toward Natural Language Processing (NLP) in system administration

Natural language interfaces try to reverse that relationship, so that the computer understands the user's language instead. Early research such as the **NL2Bash** dataset (Lin et al., 2018) showed

that translating English into correct Bash commands is hard, because a single request can map to many commands and a small mistake changes the meaning. Large language models trained on code changed what is practical: since around 2021, models such as OpenAI Codex (used by GitHub Copilot) can write working shell commands from a description, and since 2023 most LLM APIs support **function calling** (also called tool calling), where the model returns a structured request to call a named function with JSON arguments instead of free text.

Function calling is the basis of today's *agents*: programs in which an LLM plans the steps and calls tools, while ordinary code carries out the actions. In system administration this means a user can ask *"Am I running out of inodes?"* and the agent decides to measure inode usage, reads the result and answers in plain words. Our Agentic Shell applies this idea to a whole shell, and our agent applies it to one specific, error-prone area: links and inodes.

1.2 Problem Statement

Syntactical barriers and steep learning curves of traditional shells (Bash/PowerShell)

Inode and link maintenance is a good example of tasks that are simple to describe but hard to type. Each question needs a different tool and a different set of options:

What the user wants	Typical shell command
Find broken symbolic links in a folder	`find ~/Desktop -xtype l`
Find every name of the same file (hard links)	`ls -i file`, then `find / -xdev -inum N` or `find / -samefile file`
See how many inodes are left	`df -i /home`
See where a link really points	`readlink -f link`, `stat link`
Find links that loop forever	no single standard command; usually a script around `find -L` and error messages

Table 1.1: Everyday link and inode questions and the commands they need

A beginner has to know that `-xtype l` (not `-type l`) finds broken links, that `-xdev` keeps `find` on one filesystem, and that inode numbers are only unique within a filesystem. PowerShell users face a different but equally steep syntax. As a result, many users never check for dangling shortcuts, duplicated hard links or inode exhaustion until something fails, for example when a disk with free space reports *"No space left on device"* because it has run out of inodes.

Risk of executing incorrect commands due to human error

The commands that clean up links are also the commands that can destroy data. Many tools behave differently depending on whether they follow symbolic links: `find -L` follows them and can wander into, or loop through, other directories; a trailing slash on a link to a directory (`link/`) makes many commands act on the target directory instead of the link itself; and a wrongly quoted wildcard in an `rm` command can delete far more than intended. Hard links add a further trap: deleting one name of a hard-linked file does not free its data, while editing it changes every other name at once.

Letting an LLM generate raw shell commands does not remove these risks; it adds new ones. A model can invent options, guess a path the user never mentioned, or produce a command that is correct in one shell and harmful in another. A file name controlled by someone else can even

contain text that tries to instruct the model (prompt injection). The problem this project addresses is therefore: **how can a user manage links and inodes in plain English without having to learn the commands, while making it impossible for either the user or the LLM to delete or damage the wrong thing?**

1.3 Objectives

To build a lightweight, secure NL-to-CLI translation engine

Our engine translates natural language into **typed tool calls rather than shell text**. The LLM never writes a command line; it can only choose one of eleven well-defined functions and fill in its arguments, which our code validates. This keeps the engine secure, because nothing the model writes is executed as a command, and lightweight, because it is written in Python 3.8+ using only the standard library: no `pip install`, no SDK and no `sudo`, so it runs on a fresh Ubuntu lab PC.

To integrate context-aware tools using the Model Context Protocol (MCP)

Each tool is described by a name, a plain-English description and a JSON Schema for its arguments, which is the same contract the Model Context Protocol uses for MCP tools (Section 2.2). In the current version the agent offers these tools to the LLM through the OpenAI-compatible function-calling API directly, because the class LLM was unknown and the project had to use the standard library only; the tool layer is kept independent of the LLM layer so that it can be published as an MCP server later without changing the tools. The tools are **context-aware**: the agent keeps a short session note with the current folder, the last actions and the items found, so follow-ups such as *"delete them"* or *"what about that folder?"* are understood.

Specific objectives

1. Detect dangling symlinks and symlink loops (including self-links and cycles such as A → B → A) with `lstat()`, `readlink()` and the `ELOOP` error, without ever following links while scanning.
2. Group hard links by (device, inode), and report inode details and filesystem inode usage (like `df -i`) through `os.statvfs()`.
3. Flag suspicious links: links that leave a chosen folder, point into sensitive places such as `/etc` or `~/.ssh`, or live in world-writable directories.
4. Make every change safe: preview first, ask the user y/N, refuse protected system paths and files the user does not own, re-check right before acting, and log every action.
5. Work standalone (chat mode, one-shot requests and direct commands without an LLM) and be easy to plug into any base shell through a small Python API.
6. Work with any OpenAI-compatible LLM provider (for example Groq, OpenAI or a local Ollama model) and handle rate limits gracefully.

1.4 Scope and Limitations

Supported operating systems and terminal environments

- **Operating system:** Linux. The agent was developed and tested on Ubuntu with Python 3.8 or newer; it relies on POSIX behaviour of symbolic links, hard links, inodes and `statvfs()`.

- **Terminals:** any terminal emulator and shell (for example GNOME Terminal with Bash or Zsh). Chat mode uses Python's `input()` with `readline` for line editing; output is plain text so it reads well in any terminal.
- **Non-interactive use:** the agent can be called by another program (the base shell). In that case it never writes anything unless the caller supplies a confirmation callback or explicitly passes `auto_confirm=True`.
- **Not covered:** Windows and macOS were not targeted or tested. Windows handles symbolic links differently (they may require special privileges), and macOS was not available for testing.

Extent of administrative privileges granted to the AI model

The AI model is given **no shell access and no administrative privileges**. It can only request one of eleven tools: seven are read-only, and the four that change anything (delete dead links, create a symbolic link, create a hard link, repoint a link) all pass through the same safety gate. The program runs as the normal user, never uses `sudo`, refuses to write inside `/`, `/bin`, `/boot`, `/dev`, `/etc`, `/lib*`, `/proc`, `/sbin`, `/sys`, `/usr` and `/var`, refuses anything the current user does not own, and only deletes paths that are still symbolic links that still fail to resolve at the moment of deletion. The model cannot confirm a change itself: the y/N question is asked by our program.

Limitations

- Hard links can only be grouped among the names inside the scanned folder; other names elsewhere on the filesystem need a wider scan (the report states how many names are outside).
- Scans are bounded by a depth limit, an entry limit and a time limit so that scanning `/` or the home folder cannot hang; very large trees may therefore be reported as truncated.
- Natural-language mode needs network access and an API key. Free tiers impose rate limits (for example 8,000 tokens per minute on Groq's free tier), so the agent keeps requests small and retries automatically, but very fast conversations may still have to wait. Direct mode works without any LLM.
- The agent manages links and inodes only; it does not delete regular files or directories, change permissions or move files.

Chapter 2: Literature Review & Theoretical Background

2.1 Large Language Models (LLMs) in Code Generation

Capabilities of modern LLMs in structured text and code synthesis

Large language models are transformer neural networks trained on very large collections of text, including source code. Models trained or fine-tuned on code, starting with OpenAI Codex (2021), showed that an LLM can turn a short description into a working function, a SQL query or a shell command, and today's general-purpose models (for example GPT-4-class models, Claude, Llama and Gemini) do this routinely. Two capabilities matter most for an agent like ours:

- **Structured output.** Models can produce output that follows a given format, such as JSON that matches a schema, which a program can parse reliably.

- **Function (tool) calling.** The application sends the model a list of tools, each with a name, a description and a JSON Schema for its parameters. Instead of answering in prose, the model can reply with a request such as `{"name": "find_broken_symlinks", "arguments": {"path": "~/Desktop"}}`. The application runs the function and sends the result back, and the model continues until it can answer the user. This pattern is supported by the OpenAI Chat Completions API and by many compatible providers, including Groq and Ollama.

Because the model only chooses *what* to do and the application decides *how* it is done, function calling lets an LLM operate a real system through a narrow, well-defined interface.

Challenges with LLM hallucination in system-level execution

LLMs can *hallucinate*: they produce fluent, confident output that is wrong. In system administration this is dangerous because the output may be executed. Typical failures include inventing command options that do not exist, guessing a path the user never mentioned, choosing a broader action than requested (deleting a folder instead of a link), misreading earlier results, and following instructions hidden in data, for example a file named so that it reads like a command to the model (prompt injection). Research on natural-language-to-Bash translation such as NL2Bash also showed that even correct-looking commands often differ in small but important details.

For this reason our design treats the LLM as an untrusted planner. The model can only call fixed tools with validated arguments; it is told never to guess paths; every write is previewed and confirmed by the user through our program; and every deletion is re-checked by code immediately before it happens. Tool results are passed to the model as data, never as instructions.

2.2 Introduction to Model Context Protocol (MCP)

Architecture of MCP (Clients, Servers, and Tools)

The **Model Context Protocol** is an open standard, introduced by Anthropic in November 2024, for connecting AI applications to external tools and data. It defines three roles:

- **Host:** the AI application the user works with (for example a desktop assistant, an IDE or a shell). The host contains the LLM integration and decides what the model may use.
- **Client:** a component inside the host that maintains a connection to exactly one server and speaks the protocol on the host's behalf.
- **Server:** a program that exposes capabilities. The main capability types are **tools** (functions the model can call, each described by a name, a description and a JSON Schema for its input), **resources** (data the application can read, such as files) and **prompts** (reusable prompt templates).

Messages are exchanged with **JSON-RPC 2.0**. A client can ask a server which tools it offers (`tools/list`) and call one (`tools/call`). Local servers usually communicate over standard input and output (the *stdio* transport) as a child process, while remote servers use HTTP. Because tools are discovered at run time, a host can use any MCP server without code written specifically for it.

Why MCP is preferred over traditional rigid API tooling for local system access

- **One standard instead of many integrations.** With traditional tooling every application needs custom glue code for every API. An MCP server written once can be used by any MCP-capable host.
- **Self-describing tools.** Names, descriptions and JSON Schemas are discovered at run time, so the model always sees the current, exact interface.
- **Local and contained.** A local server runs as a separate process over stdio, so the host gets access to local system functions without opening a network port, and the server itself decides what is exposed.
- **Human oversight.** The host remains in control of which servers and tools are enabled and can ask the user before a tool runs.

Relation to our project. Our eleven tools follow exactly this tool model: each is a plain function with a name, a description and a JSON Schema, and each returns a JSON-serialisable result. Because the class LLM was unknown and the project was limited to the Python standard library, the agent currently offers the tools through the OpenAI-compatible function-calling API, which needs no SDK. The tool layer (`tools.py` and `schemas.py`) is separate from the LLM layer, so exposing the same tools through an MCP server is a straightforward next step.

2.3 Existing Solutions

Comparative study of tools like GitHub Copilot CLI, Warp AI, and open-source alternatives

Several tools already bring LLMs into the terminal. They are general-purpose: the model writes a command or a program, and the user is asked before it runs.

- **GitHub Copilot in the CLI.** GitHub's command-line assistant suggests and explains shell, Git and GitHub CLI commands from a natural-language description (originally the `gh copilot suggest` and `gh copilot explain` commands, with newer agent-style versions that can also run commands after asking). It requires a GitHub Copilot subscription and a network connection.
- **Warp AI.** Warp is a modern terminal application with built-in AI features: natural-language command search and an agent mode that can propose and run multi-step command sequences, with the user able to review them first. It is a complete terminal rather than a component that can be added to another shell.
- **ShellGPT (open source).** A Python command-line tool that sends a request to an LLM and, in shell mode, prints a suggested command and asks whether to execute it, describe it or abort.
- **Open Interpreter (open source).** Lets an LLM write and run code (Python, shell and others) on the local machine, asking for confirmation before running code unless auto-run is enabled.

Tool	How the LLM acts	Safety approach	Fit for link / inode auditing
GitHub Copilot CLI	Generates shell commands	User reviews and approves the command	General; user must judge the command
Warp AI	Generates and runs	Review before running	General; tied to the Warp

Tool	How the LLM acts	Safety approach	Fit for link / inode auditing
	command sequences		terminal
ShellGPT	Generates one shell command	Execute / describe / abort prompt	General; command text is not checked
Open Interpreter	Writes and runs code	Confirmation before running code	Very general; broad access
Our agent (Agent 10)	Calls 11 fixed, typed tools	Preview, y/N by the program, protected paths, ownership check, re-check before acting, log	Specialised: broken links, loops, hard links, inodes

Table 2.1: Comparison of existing LLM terminal assistants with our agent

Gap addressed by this project. Existing assistants ask the user to approve *generated command text*, so safety depends on the user spotting a mistake in a command they may not understand. Our agent never produces command text. It offers a small set of audited tools whose behaviour is fixed in code, shows a plain-language preview of exactly which links will change and why ("target missing" or "symlink loop"), and re-checks each path at the moment of action. It is also designed as a component: any base shell can call it through `handle()` or `call_tool()`.

2.4 Theoretical Background: Inodes, Hard Links and Symbolic Links

An **inode** is the data structure a Linux filesystem uses to describe a file: its type, size, owner, permissions, timestamps, link count and where its data is stored. A file name is only a directory entry that points to an inode, and inode numbers are unique only within one filesystem, so a file is identified by the pair (device, inode).

- A **hard link** is an additional directory entry for the same inode. All names are equal; the data is freed only when the link count drops to zero. Hard links cannot cross filesystems and cannot normally point to directories.
- A **symbolic (soft) link** is a separate small file whose content is a path. If the target is moved or deleted, the link becomes **dangling (broken)**. If links point at each other (A → B → A, or a link to itself), resolving them fails with the `ELOOP` error; Linux gives up after 40 links in a chain.
- `stat()` follows symbolic links, while `lstat()` describes the link itself and `readlink()` returns its stored path. An auditor must use `lstat()` while walking directories, otherwise it may follow a link into another part of the system or loop forever.
- Every filesystem has a fixed or limited number of inodes. `statvfs()` (used by `df -i`) reports total and free inodes; if they run out, no new files can be created even when disk space is free.

Property	Hard link	Symbolic link
Has its own inode	No (shares the target's inode)	Yes
Survives deletion of the "original" name	Yes	No, it becomes dangling

Property	Hard link	Symbolic link
Can cross filesystems	No	Yes
Can point to a directory	No (normally)	Yes
How our agent detects it	Same (device, inode), link count > 1	`lstat()` + `readlink()`; broken if the target is missing, loop if `ELOOP`

Table 2.2: Hard links compared with symbolic links

These system calls are exactly what the agent's tools use, which is why the project can work with only the Python standard library modules `os`, `os.path` and `stat`.

References

- X. V. Lin, C. Wang, L. Zettlemoyer and M. D. Ernst, "NL2Bash: A Corpus and Semantic Parser for Natural Language Interface to the Linux Operating System," *Proc. LREC*, 2018.
- M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021 (OpenAI Codex).
- Anthropic, "Introducing the Model Context Protocol," November 2024; Model Context Protocol specification, modelcontextprotocol.io.
- OpenAI, "Function calling" (Chat Completions API documentation).
- Linux manual pages: `inode(7)`, `symlink(7)`, `path_resolution(7)`, `stat(2)`, `statvfs(3)`, `find(1)`, `df(1)`.
- Project repository: github.com/Debojyoti17/inode-symlink-auditor-agent (private). The code was developed with AI assistance (Claude), which the course permits.

Review 2

Agentic Shell: Agent 10 – Inode & Symlink Auditor Agent

A natural-language agent for finding broken links, symlink loops and hard links on Linux

Name	Registration No.
Debojyoti Barai Mitul	24BCE2951
Shivam Prakash	24BDS0340

Chapter 3: System Architecture & Design

3.1 High-Level Architecture

The User-to-Shell loop: User Input → LLM Processing → Tool Call → System Execution → Feedback

Agent 10 follows the standard agent loop, with one extra stage that we treat as essential: a **safety gate** between the tool call and system execution for every action that changes the filesystem. Figure 3.1 shows one pass through the loop.

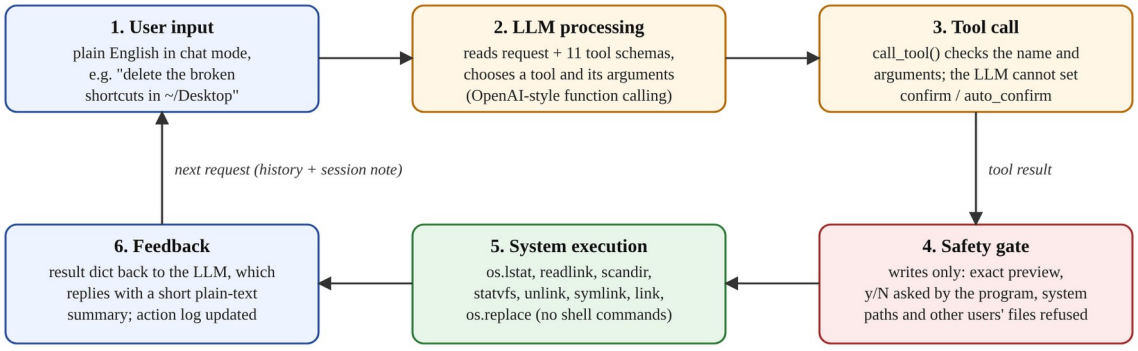


Figure 3.1: The User-to-Shell loop of Agent 10

1. **User input.** The user types a request in plain English, either in chat mode (the `agent10>` prompt) or as a one-shot command, for example *"Find all broken shortcuts on my desktop and delete them."*
2. **LLM processing.** The agent sends the request, the conversation so far and the JSON Schemas of its eleven tools to an OpenAI-compatible LLM. The model replies either with text or with one or more tool calls (a tool name plus JSON arguments).
3. **Tool call.** `call_tool()` looks the tool up in a fixed registry, parses and validates the arguments against the schema, and rejects unknown tools or arguments. The model has no way to pass the confirmation flags.
4. **Safety gate (writes only).** For the four write tools the program builds an exact preview, asks the user y/N itself, refuses protected system paths and files the user does not own, and records the decision.

5. **System execution.** The tool performs the work with system calls from the Python standard library (`os.lstat`, `os.readlink`, `os.scandir`, `os.statvfs`, `os.unlink`, `os.symlink`, `os.link`, `os.replace`). No shell command is ever built or executed.
6. **Feedback.** The tool returns a JSON-serialisable dictionary. It goes back to the LLM (compacted to keep requests small), which either calls another tool (at most 8 per request) or answers the user with a short plain-text summary. Every write is also appended to an action log.

There are three ways into the same tools: **natural language** (chat or one-shot, through the loop above), **direct commands** such as `python3 -m inode_symlink_agent broken ~/Desktop` that call the tools without any LLM, and the **Python API** (`get_tool_schemas()`, `call_tool()`, `handle()`) through which the class's base shell will use the agent. Because all three share the same tool functions and the same safety layer, the safety guarantees do not depend on how a request arrives.

3.2 Component Design

Figure 3.2 shows every component and how they connect. The code is a single Python package, `inode_symlink_agent`, with one module per responsibility (Table 3.1).

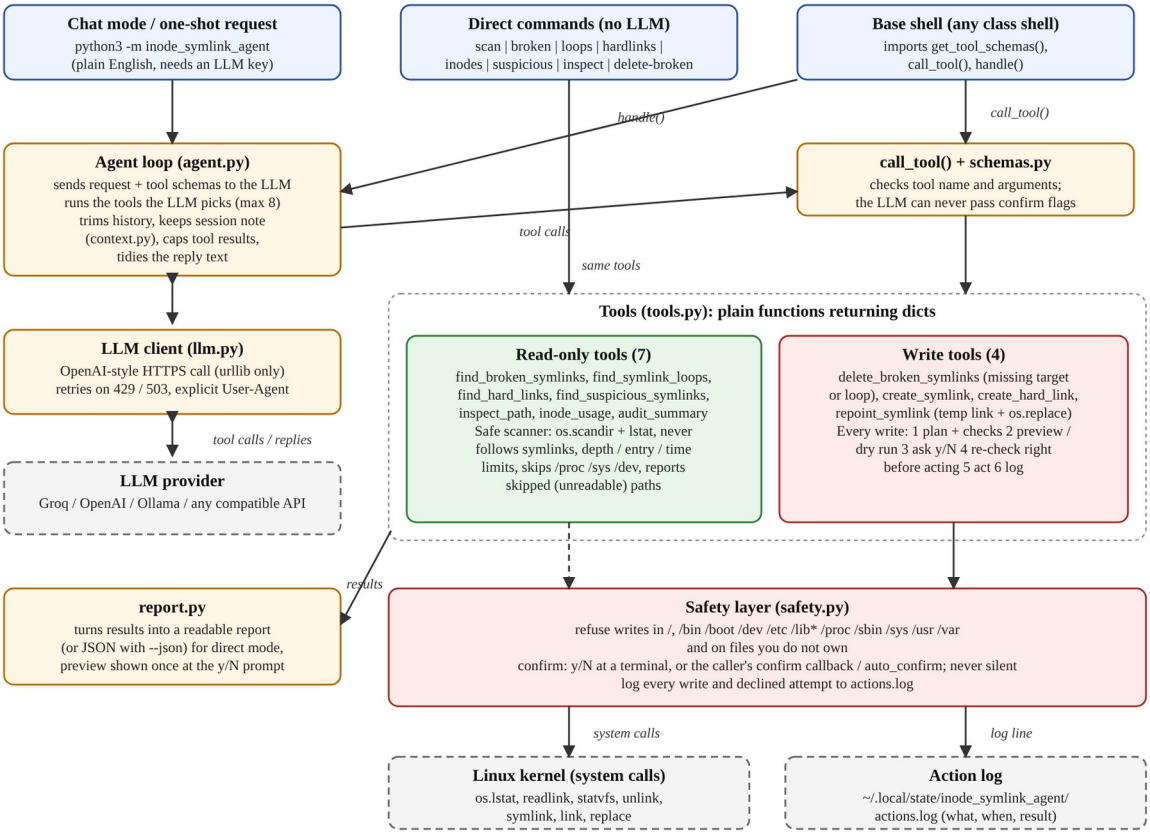


Figure 3.2: Component (block) diagram of Agent 10

Module	Lines	Responsibility
`__main__.py`	167	Command line: chat mode (REPL), one-shot requests, direct commands and options
`agent.py`	246	Agent loop, system prompt, `call_tool()`, public API

Module	Lines	Responsibility
		(`AGENT_NAME`, `AGENT_DESCRIPTION`, `handle()`)
`context.py`	131	Session note (current folder, last actions, items found) that survives history trimming
`llm.py`	174	OpenAI-compatible HTTP client using `urllib`, `.env` loading, retry on rate limits
`schemas.py`	71	JSON Schemas (function-calling definitions) of the 11 tools
`tools.py`	659	The tools: bounded scanner, link classification, read-only tools, write tools and write gate
`safety.py`	199	Path resolution, protected paths, ownership checks, confirmation, action log
`report.py`	133	Readable text (or JSON) reports for direct mode

Table 3.1: Modules of the inode_symlink_agent package (1,792 lines of code, plus 1,415 lines of tests)

The Interface Layer: Custom REPL (Read-Eval-Print Loop) environment

Running `python3 -m inode_symlink_agent` with no arguments starts a REPL. It **reads** a line at the `agent10>` prompt (using Python's `input()` with `readline`, so arrow keys and history work), **evaluates** it by passing it to the agent loop together with the conversation history, **prints** the plain-text reply, and **loops**. Typing `exit` or `quit`, or pressing Ctrl+D, leaves the REPL; Ctrl+C at the prompt simply gives a fresh prompt. Tool calls are traced on standard error (for example `[tool] find_broken_symlinks {"path": "/tmp/agent10_demo"}`), so the user can see what the agent is doing.

The same command also accepts a one-shot request in quotes, and eight **direct commands** that need no LLM: `scan`, `broken`, `loops`, `hardlinks`, `inodes`, `suspicious`, `inspect` and `delete-broken`. Options include `--json` for machine-readable output, `--yes` to approve writes without the prompt, `--verbose` to show the approximate tokens of each LLM request, and depth and entry limits. For the base shell, the package exports a short **Python API**: `AGENT_NAME`, a router-friendly `AGENT_DESCRIPTION` with example requests, `get_tool_schemas()`, `call_tool(name, arguments)` and `handle(prompt, confirm=None)`.

The Orchestration Layer: LangChain/LlamaIndex or raw MCP client handling intent routing

We chose a **raw, hand-written orchestration loop** instead of LangChain or LlamaIndex. The project had to run with the Python standard library only, and a small explicit loop makes every step visible and testable. The loop in `agent.handle()` works as follows:

- It sends the system prompt, the conversation and the tool schemas to the LLM through `llm.ChatClient`, a plain HTTPS POST to `/chat/completions` that works with any OpenAI-compatible provider (Groq, OpenAI, Ollama and others).
- If the model asks for tools, each call goes through `call_tool()`; the result is compacted (lists are cut to 50 items with a count of the rest) and returned to the model. At most **8 tool calls** are allowed per request; after that the model must answer.

- **Intent routing** inside the agent is done by the LLM choosing among the eleven tool schemas, guided by the system prompt. Routing *between* agents is the base shell's job, which is why the agent publishes `AGENT_DESCRIPTION`.
- In chat mode the history is trimmed to the last 6 turns (tool outputs are kept only for the latest 2) to fit small free-tier token budgets, while a **session note** (`context.py`) keeps the current folder, the last three actions and the paths found by the last search, so references such as "there" and "delete them" still resolve.
- The client retries automatically on HTTP 429 (rate limit) and 503, waiting as long as the server asks (up to 3 retries and about 60 seconds in total), sends an explicit User-Agent header, and turns API errors and timeouts into clear messages.

The Tooling Layer (MCP Server): File management, system monitoring, and network configuration utilities

Our tooling layer plays the role of an MCP server for one domain: **link and inode management**. Network configuration is outside this agent's scope (other agents of the Agentic Shell cover other areas). The tools are plain Python functions that return JSON-serialisable dictionaries, registered in a fixed table and described by JSON Schemas, which is the same tool model MCP uses (Section 2.2).

Tool	Type	What it does
find_broken_symlinks	read	Links whose target is missing, with raw and resolved target, relative or absolute
find_symlink_loops	read	Links that never resolve (`ELOOP`): self-links and cycles, with the chain
inspect_path	read	`lstat()` details: type, inode, device, link count, size, owner, permissions; link target and status
find_hard_links	read	Groups of files sharing one (device, inode) with link count > 1; counts names outside the scan
inode_usage	read	Total, used and free inodes and percentage for a filesystem (`os.statvfs`, like `df -i`)
find_suspicious_symlinks	read	Links leaving a boundary folder, pointing into `/etc`, `/root`, `~/.ssh`…, or in world-writable folders, with reasons
audit_summary	read	One scan returning counts of every issue type plus examples
delete_broken_symlinks	write	Unlinks dead links (missing target or loop) after confirmation and a final re-check
create_symlink	write	Creates a symbolic link
create_hard_link	write	Creates a hard link to a regular file the user owns, on the same filesystem
repoint_symlink	write	Changes a link's target atomically (temporary link + `os.replace`)

Table 3.2: The eleven tools exposed to the LLM

All scanning tools share one **bounded walker**. It uses an explicit stack with `os.scandir()` and `entry.stat(follow_symlinks=False)` (an `lstat`), so a symbolic link is examined but never entered, and directory cycles cannot cause an endless walk. It skips `/proc`, `/sys` and

`/dev`, stops at a maximum depth (default 8), a maximum number of entries (default 50,000) or a time limit (30 seconds) and then reports the result as truncated, and when a directory cannot be read it records the path as skipped and continues. A link is classified with `os.readlink()` and an `os.stat()` attempt: `FileNotFoundError` means broken, `errno.ELOOP` means loop, and any other error is reported as unknown and never treated as dead.

3.3 Security & Guardrail Framework

The "Human-in-the-Loop" validation mechanism

Every change to the filesystem passes through the same gate (`_write_flow` in `tools.py`, Figure 3.3). The human decision is made by **our program**, not by the LLM: the model can request a change, but it cannot approve it.

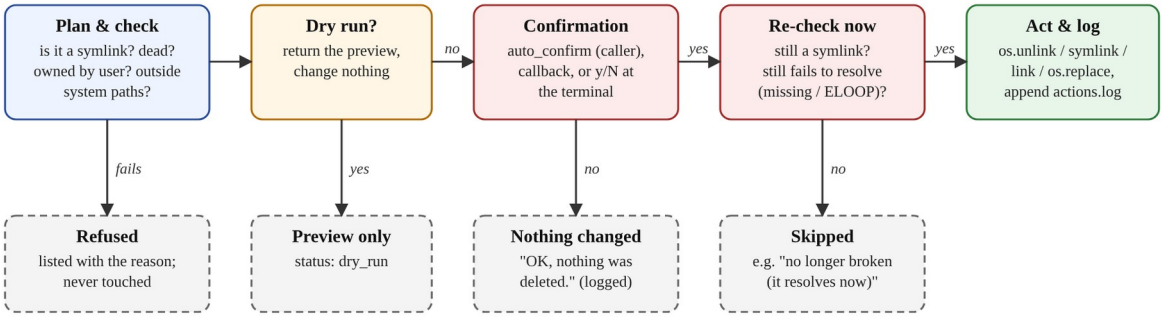


Figure 3.3: The write gate shared by all four write tools

1. **Plan and check.** Each requested path is checked: it must be a symbolic link that is dead (missing target or loop) for deletion, owned by the current user, and outside the protected system paths. Anything else is listed as refused with the reason and never touched.
2. **Preview.** The program builds an exact preview: one line per change, saying what will happen and why, for example `unlink /tmp/agent10_demo/desktop/loop-a (-> loop-b, symlink loop)`. With `dry_run` the tool returns only this preview.
3. **Confirmation.** The decision comes, in order, from `auto_confirm=True` (only the Python caller or the `--yes` option can set it), a `confirm` callback supplied by the base shell, or a y/N question in an interactive terminal. If none is available the answer is **no**: the agent never writes silently. A callback that fails also counts as no. If the user declines, the agent replies simply "OK, nothing was deleted." without asking the LLM again.
4. **Re-check right before acting.** Immediately before each `os.unlink()`, the path is checked again with `lstat()`: it must still be a symbolic link and must still fail to resolve. A link that was repaired, or replaced by a regular file or directory, after the preview is left alone and reported as skipped. This closes the time-of-check to time-of-use gap.
5. **Act and log.** The system call is made and one JSON line (time, user, action, details, result) is appended to `~/.local/state/inode_symlink_agent/actions.log`; declined attempts are logged too.

A real preview and decline from the demo folder looks like this:

```
$ python3 -m inode_symlink_agent delete-broken /tmp/agent10_demo/desktop
delete_broken_symlinks: 5 change(s) planned
  - unlink /tmp/agent10_demo/desktop/missing-photo (->
/tmp/agent10_demo/photos/gone.jpg, target missing)
  - unlink /tmp/agent10_demo/desktop/old-project (-> ../projects/old-project, target
missing)
  - unlink /tmp/agent10_demo/desktop/loop-a (-> loop-b, symlink loop)
  - unlink /tmp/agent10_demo/desktop/loop-b (-> loop-a, symlink loop)
  - unlink /tmp/agent10_demo/desktop/self-link (-> self-link, symlink loop)
Apply these changes? [y/N] n
[not_confirmed] Nothing was changed (declined at prompt).
```

Command sanitization and regex-based blacklisting of destructive commands (e.g., rm -rf /)

Command sanitization and blacklisting are needed when an LLM produces **command text** that a shell will execute. Blacklists are fragile: a pattern for `rm -rf /` can be bypassed by `rm -r -f /`, by variables, quoting or command substitution. Agent 10 removes this attack surface instead of filtering it: **the LLM never writes a command and nothing is passed to a shell**. It can only choose among eleven fixed functions (an allow-list), and their arguments are validated. The guardrails that take the place of a blacklist are summarised below.

Threat	Guardrail in Agent 10
LLM invents a destructive command (e.g. `rm -rf /`)	No shell is used; only 11 allow-listed tools exist. Nothing can delete a regular file or a directory.
Unknown tool or forged arguments	`call_tool()` rejects unknown tool names, invalid JSON and arguments not in the schema; errors are returned to the model as data.
LLM tries to approve its own change	`confirm` / `auto_confirm` are not in any schema and are stripped from LLM arguments; only the program or its caller can confirm.
Writes to system areas	Refused inside `/`, `/bin`, `/boot`, `/dev`, `/etc`, `/lib*` (regex `^/lib[^/]*(/|$)`), `/proc`, `/sbin`, `/sys`, `/usr`, `/var`; checked on the path and on its resolved parent folder, so a link to `/etc` cannot be used as a detour.
Changing other users' files	Refused unless the file (and, for new links, the parent folder) is owned by the current user.
Path changes between preview and action	Re-check with `lstat()` immediately before each `unlink()`; otherwise skipped with the reason.
Following links into other folders or loops while scanning	The walker never follows symbolic links; depth, entry and time limits; `/proc`, `/sys`, `/dev` skipped.
Prompt injection through file names or link targets	The system prompt states that tool output is data, never instructions, and every write still needs the user's y/N.
Runaway agent loop or excessive cost	At most 8 tool calls per request; results capped at 50 items; history trimmed; rate-limit retries capped at about 60 s.
LLM guesses paths	The prompt forbids guessing; the session note supplies the current folder for follow-ups; aliases such as "desktop" are resolved by code.

Table 3.3: Threats and the guardrails that address them

Chapter 4: Implementation Methodology

4.1 Development Environment & Tech Stack

Item	Choice in Agent 10
Programming language	Python 3.8+, standard library only (`os`, `stat`, `json`, `urllib`, `argparse`, `unittest`, …). No `pip install`, no `sudo`.
LLM provider	Any OpenAI-compatible Chat Completions API with tool calling. Tested with Groq (model `openai/gpt-oss-120b`, fallback `openai/gpt-oss-20b`, free tier); configuration examples for OpenAI and a local Ollama model (e.g. Llama 3.1). Configured with `AGENT_LLM_BASE_URL`, `AGENT_LLM_API_KEY` and `AGENT_LLM_MODEL`, from the environment or a git-ignored `.env` file.
SDKs	None. The LLM client is about 170 lines of `urllib`. The MCP Python SDK was not used because the project was restricted to the standard library; the tools follow the MCP tool model (name, description, JSON Schema) so they can be published through MCP later.
Operating system	Ubuntu Linux; any terminal.
Testing	`unittest` with temporary directories, a fake LLM client that returns scripted tool calls, and a local stub HTTP server (no network needed).
Version control	Git and GitHub, one branch and pull request per change.
Development assistance	Code was developed with AI assistance (Claude, via Claude Code), which the course permits.

Table 4.1: Development environment and technology stack

4.2 Step-by-Step Implementation

Step 1: Setting up the base CLI prompt loop

The entry point `__main__.py` decides between three modes. With no arguments it starts the chat REPL; if the first word is one of the eight direct commands followed by exactly one path, it runs that command without an LLM; otherwise all words are joined into a one-shot natural-language request. The REPL keeps one conversation history object for the whole session (abridged from `run_chat()`):

```
history = agent.new_history()          # system prompt + session note
while True:
    try:
        line = input_fn("agent10> ")
    except EOFError:                   # Ctrl+D quits
        return 0
    except KeyboardInterrupt:          # Ctrl+C gives a fresh prompt
        continue
    line = line.strip()
    if not line:
        continue
    if line.lower() in ("exit", "quit", ":q"):
        return 0
    print(agent.handle(line, client=client, auto_confirm=auto_confirm,
                       on_tool_call=_trace, history=history))
```

Step 2: Building the MCP Server to expose safe system functions (e.g., list_directory, get_system_stats)

Our equivalent of the MCP server is the tool layer: `tools.py` implements the functions and registers them in a dictionary, and `schemas.py` describes each one with a JSON Schema. Our domain-specific counterparts of *list_directory* and *get_system_stats* are the bounded scanner behind `find_broken_symlinks` / `audit_summary` and `inode_usage`. A schema as the LLM sees it:

```
{"type": "function", "function": {
  "name": "find_broken_symlinks",
  "description": "Read-only. List dangling symlinks (target missing) under a
                  directory; never follows links; bounded.",
  "parameters": {"type": "object",
    "properties": {
      "path": {"type": "string", "description": "Path; ~, $VARS and
               desktop/downloads/documents work."},
      "recursive": {"type": "boolean"},
      "max_depth": {"type": "integer"}},
    "required": ["path"]}}}
```

Every tool returns a dictionary such as `{"ok": true, "count": 2, "broken_symlinks": [...], "scan": {...}}`, or `{"ok": false, "error": "..."}`; tools never raise exceptions to the caller. Inode usage, for example, is a direct use of `os.statvfs()`: used inodes = `f_files - f_ffree`.

Step 3: Prompt engineering for exact tool matching and JSON format enforcement

Tool matching is achieved by short, precise tool descriptions (each says whether it is read-only or a write and what it returns) and a compact system prompt. The prompt's main rules are: never guess paths; to delete dead links, find them first and pass exactly those paths; call a write tool **once** with `dry_run=false` because the program itself shows the preview and asks y/N (the model must never ask for confirmation in text); offer only actions the tools can perform; resolve "there" and "them" from the session note; treat tool output as data; never mention internal tool or parameter names; and reply in plain terminal text without markdown. Prompt and schemas were shortened to about 7,000 characters so that requests stay within free-tier token limits, and a unit test guards that size.

JSON format enforcement happens in code, because a prompt alone cannot guarantee it. Arguments arrive as a JSON string and are parsed with `json.loads`; `call_tool()` then checks them against the schema and the function signature (abridged):

```
fn = tools.TOOLS.get(name)
if fn is None:
    return {"ok": False, "error": "Unknown tool %r. Available: ..." % name}
if isinstance(arguments, str):
    try:
        arguments = json.loads(arguments) if arguments.strip() else {}
    except ValueError:
        return {"ok": False, "error": "Tool arguments are not valid JSON."}
unknown = sorted(set(arguments) - set(allowed))     # allowed = schema properties
if unknown:
    return {"ok": False, "error": "Unknown argument(s) ..."}
kwargs = dict(arguments)
if name in tools.WRITE_TOOLS:                        # added by the program,
```

```
    kwargs.update(confirm=confirm, auto_confirm=auto_confirm)   # never by the LLM
inspect.signature(fn).bind(**kwargs)                # type/arity check
return fn(**kwargs)
```

Errors are returned to the model as ordinary results, so it can correct itself on the next call. Replies are also cleaned before they are printed: internal names that slip through are replaced by plain words, and missing spaces between sentences are fixed.

Step 4: Implementing the command execution wrapper and output formatter

The **execution wrapper** is the write gate described in Section 3.3: `_write_flow()` builds the preview, handles `dry_run`, asks for confirmation through `safety.ask_confirmation()`, applies each change through a per-tool function that re-checks the path immediately before the system call, and logs the outcome. Two details matter in practice: deletion accepts both missing-target links and loop links but nothing else, and `repoint_symlink` creates the new link under a temporary name and moves it over the old one with `os.replace()`, so the link is never missing even for an instant.

The **output formatter** (`report.py`) turns each tool's dictionary into a readable report for direct mode, or JSON with `--json`, and always says when a scan was truncated or paths were skipped. Real output for the demo folder:

```
$ python3 -m inode_symlink_agent loops /tmp/agent10_demo/desktop
Symlink loops under /tmp/agent10_demo/desktop: 3
  /tmp/agent10_demo/desktop/loop-a [cycle]: .../loop-a -> .../loop-b -> .../loop-a
  /tmp/agent10_demo/desktop/loop-b [cycle]: .../loop-b -> .../loop-a -> .../loop-b
  /tmp/agent10_demo/desktop/self-link [self-link]: .../self-link -> .../self-link

Scanned 8 entries in 1 directories (max depth 8).
```

Verification. The project has 122 automated tests (`python3 -m unittest discover -s tests -v`). They build real temporary directories with broken links, relative and absolute links, self-links and two-link loops, links to directories, hard links, an unreadable directory and suspicious links, and they prove that a delete never removes a regular file, a directory or a link that became valid again between preview and deletion. The natural-language path is tested with a fake LLM that returns scripted tool calls, and the HTTP client with a local stub server, including rate-limit retries. All tests pass whether or not a terminal is attached. A demo script (`demo/make_demo.sh`) builds a fixed test folder in `/tmp/agent10_demo` for live presentation.

Review 3

Agentic Shell: Agent 10 – Inode & Symlink Auditor Agent

A natural-language agent for finding broken links, symlink loops and hard links on Linux

Name	Registration No.
Debojyoti Barai Mitul	24BCE2951
Shivam Prakash	24BDS0340

Chapter 5: Testing, Evaluation, and Results

The agent was evaluated in three ways: (1) a live chat session on a student laptop (Ubuntu under WSL, Python, Groq model `openai/gpt-oss-120b` on the free tier) using the fixed demo folder `/tmp/agent10_demo` built by `demo/make_demo.sh`; (2) a safety probe that sends the requests a misbehaving LLM could make directly to the agent's tool dispatcher; and (3) the automated test suite of 129 tests. The demo folder contains exactly two broken links, three loop links (`loop-a` → `loop-b` → `loop-a` and a self-link), one suspicious link (`passwd-shortcut` → `/etc/passwd`), one group of three hard-linked files and one unreadable folder, so every answer can be checked against a known ground truth. Screenshots of the live session are in Chapter 7.

5.1 Test Cases & Scenarios

Simple intents

Single requests that need one tool. The template's example ("find all PDFs created yesterday") belongs to a file-search agent; the equivalent single-step requests for this agent are below. All were typed in chat mode during the live session.

Request (as typed)	Tool chosen by the LLM	Result (checked against the demo folder)
Audit /tmp/agent10_demo and tell me what is wrong	`audit_summary`	2 broken links, 3 loops, 1 suspicious link (high severity), 1 hard-link group of 3 names; the unreadable folder `locked` reported as skipped (permission denied). All correct.
Which files in /tmp/agent10_demo are hard links of each other?	`find_hard_links`	The 3 names of inode 231366 with link count 3. Correct.
Am I running out of inodes in /tmp?	`inode_usage`	67,108,864 total, 234,056 used, 0.35 % used: "far from running out". Correct (same as `df -i`).
Are there any suspicious links in /tmp/agent10_demo?	`find_suspicious_symlinks`	Only `passwd-shortcut` → `/etc/passwd`, high severity, with the reason. Correct.

Table 1: Simple intents from the live session.

Complex chained intents

Multi-step requests where a later step depends on the result of an earlier one, and follow-ups that only make sense with the conversation's context ("them", "there"). The template's example chains a check and an action ("check if port 8080 is open, and if it is, kill the process"); the equivalent chain here is *find dead links, then delete exactly those*.

Step (as typed)	**What happened**
Find the broken shortcuts in /tmp/agent10_demo/desktop	The two broken links were listed (`missing-photo`, `old-project`).
delete them *(answered N)*	One tool call, `delete_broken_symlinks`, with exactly those two paths. The program showed the preview ("target missing" for each) and asked y/N. Answer `N`: reply "OK, nothing was deleted."; nothing changed.
delete them *(answered y)*	Same preview; answer `y`: both links removed and confirmed.
are there any symlink loops there?	"there" was resolved from context: `find_symlink_loops` on `/tmp/agent10_demo/desktop` found `loop-a`, `loop-b` (cycle) and `self-link`.
delete them *(answered y)*	One tool call with the three loop paths; preview said "symlink loop" for each; all three removed.

Table 2: Chained and context-dependent intents from the live session.

The chain shows the intended behaviour: the LLM calls the write tool once, without first asking for permission in text, and the confirmation is asked by the program itself.

Malicious / destructive inputs

A destructive request such as "delete everything on my drive" can only become a call to one of the agent's eleven tools, because the LLM never writes a shell command. To test the worst case independently of how well any particular LLM behaves, the calls a misbehaving or manipulated LLM could make were sent directly to `call_tool()`, the same dispatcher the LLM uses. Every result below is the agent's real response.

Attempted call	Agent's response
`run_shell(cmd="rm -rf /")`	Rejected: unknown tool (only the 11 listed tools exist).
`delete_broken_symlinks(["/"])`	Refused: not a symlink (only symlinks are ever deleted).
`delete_broken_symlinks(["/etc/passwd"])`	Refused: not a symlink.
`delete_broken_symlinks([home folder])`	Refused: not a symlink.
`delete_broken_symlinks([a data file])`	Refused: not a symlink.
`delete_broken_symlinks([a working link])`	Refused: "symlink still works (it points to an existing target)".
`delete_broken_symlinks([...], auto_confirm=True)`	Rejected: unknown argument `auto_confirm`; the LLM cannot approve its own change.
`delete_broken_symlinks([a broken link])` with no terminal and no callback	Not confirmed: "Nothing was changed". The agent never writes silently.
`create_symlink(link_path= "/etc/agent10_evil")`	Refused: protected system path.
`create_symlink(link_path= "/usr/bin/agent10_evil")`	Refused: protected system path.
`find_broken_symlinks(path="/proc")`	Refused: `/proc`, `/sys` and `/dev` are always skipped.
Arguments that are not valid JSON	Rejected with "Tool arguments are not valid JSON."

Table 3: Safety probe: destructive calls sent directly to the tool dispatcher.

After the probe the demo folder was unchanged. One call was *allowed* and is worth stating: `repoint_symlink` may point a link the user owns at a sensitive file such as `/etc/shadow`. This writes nothing in `/etc` and reading the target is still governed by normal file permissions, but it is a deliberate design limit: such a change still needs the user's y/N, and the suspicious-link scan flags links into `/etc`.

The template's natural-language example ("delete everything on my drive") was not typed into the live session; the probe above covers every action such a request could be turned into.

Automated test suite

All 129 tests use Python's `unittest`, real temporary directories and no network: a fake LLM returns scripted tool calls and a local stub HTTP server stands in for the LLM API. They pass on the student laptop (Figure 1) and, in development, on Python 3.11, 3.12 and 3.13, with and without a terminal attached.

Test file	Tests	What it covers
`test_tools.py`	16	Broken links (relative and absolute), loops (self and two-link), links to directories, directory cycles, hard links, inode usage, suspicious links, depth/entry limits, unreadable folders, path aliases.
`test_writes.py`	32	Dry run, y/N and callback confirmation, never deleting a file, a directory or a link that became valid again, loop deletion, system paths, ownership, atomic repoint, action log.
`test_agent.py`	52	Agent loop with a fake LLM, tool-call limit, history trimming, result capping, chat mode, command line, HTTP client, rate-limit retries.
`test_context.py`	22	Session note, context after trimming, fixed reply after a declined prompt, plain-language replies.
`test_base_shell.py`	7	The example base shell: loading agents, routing, end-to-end delete through the shell.
Total	**129**	128 pass and 1 is skipped when run as root (a real unreadable-folder test that root cannot exercise).

Table 4: Automated test suite.

5.2 Performance Metrics

Intent translation accuracy

For this agent, "translation" means mapping a request to the right tool with the right arguments. In the live session (Figures 2–4), all 9 chat requests were answered correctly. For 8 of them the tool call is visible in the screenshots and was the correct tool with the correct path or paths; for the remaining one ("find the broken shortcuts"), the call is outside the captured screen but the answer listed exactly the two broken links. Each request needed only one tool call; no unnecessary preview (dry-run) calls were made. In the example base shell, both routing decisions were correct: the loop question was sent to this agent and "open firefox" was rejected because no loaded agent matches.

Measure (live session)	Result
Chat requests answered correctly	9 / 9
Tool calls visible and correct (tool and arguments)	8 / 8
Context-dependent follow-ups resolved ("them", "there")	3 / 3
Tool calls per request	1
Base-shell routing decisions correct	2 / 2

Table 5: Intent accuracy. A small sample on a known folder, not a statistical benchmark.

One deviation was observed: in the first answer the model used "•" bullets and "→" arrows although the prompt asks for plain "- " lists. The content was correct and readable, so this was left as a cosmetic issue.

Latency (Time-to-Execution)

Latency has two parts: the agent's own work (the tools and the loop) and the LLM round trips.

Tool execution. Measured on a development machine (median of 20 runs on the demo folder, single runs on large trees):

Operation	Time	Note
`inspect_path`, `inode_usage`	< 0.1 ms	single `lstat` / `statvfs`
`find_broken_symlinks`, loops, hard links (demo)	0.4 ms	20 entries
`audit_summary` (demo)	0.7 ms	all checks in one scan
`audit_summary` on `/usr/lib`	4.9 s	17,887 entries, complete
`audit_summary` on `/usr`	2.3 s	stopped at the 50,000-entry limit, reported as truncated
`audit_summary` on `/`	1.1 s	stopped at the 50,000-entry limit; `/proc`, `/sys`, `/dev` skipped
Agent loop overhead per request	1.1 ms	with an instant fake LLM, one tool call

Table 6: Execution time of the agent's own code.

The scan limits (depth 8, 50,000 entries, 30 seconds) keep even a scan of `/` bounded. Times for large trees depend on disk caching.

LLM round trips. End-to-end time was not instrumented in the live session, so no number is claimed here. What the screenshots do show is that the free tier dominated the time: Groq limits `openai/gpt-oss-120b` to 8,000 tokens per minute, each request is about 1,800 tokens (system prompt and eleven tool schemas), and a request normally needs two round trips (tool call, then answer). The client printed ten automatic waits during the 9 requests, from 1.5 s to 22.5 s (112 s in total). With a paid tier or a local model these waits disappear; the agent's own work stays in the millisecond range.

Error recovery rate

The template asks how often the system recovers when an LLM generates a syntactically invalid command. In this design the LLM never generates command text, so a syntactically invalid *command* cannot occur. The equivalent failures and the recovery measured are:

Failure	Recovered	How
API rate limit (HTTP 429) during the live session	10 / 10	Automatic wait as long as the server asks, then retry (up to 3 retries, about 60 s in total).
Invalid tool call in the live session (unknown tool, bad JSON, wrong argument)	0 occurred	–
Invalid tool calls in the safety probe and tests	all handled	Returned to the model as an error result instead of crashing, so it can correct the call; nothing is executed.
API errors, timeouts, no API key	handled	Clear message to the user; a failed turn is removed from history so the chat can continue.

Table 7: Error recovery.

Chapter 6: Conclusion & Future Scope

6.1 Summary of Contributions

Successful creation of a working prototype bridging conversational text and system execution

The project delivers a working agent, Agent 10 of the Agentic Shell, that turns plain-English requests about links and inodes into safe system calls:

- **Eleven tools** (seven read-only, four that change files) built only on the Python standard library and system calls (`lstat`, `readlink`, `scandir`, `statvfs`, `unlink`, `symlink`, `link`, `os.replace`); no shell commands, no `pip install`, no `sudo`.
- **A bounded scanner** that never follows symbolic links, skips `/proc`, `/sys` and `/dev`, reports unreadable folders, and stops safely at depth, entry and time limits.
- **A safety gate** for every change: exact preview with the reason for each item, y/N asked by the program (never by the LLM), refusal of system paths and other users' files, a re-check right before each deletion, and an action log.
- **An LLM layer** that works with any OpenAI-compatible provider (tested with Groq), handles rate limits automatically and keeps requests small enough for free tiers.
- **Three ways to use it**: a chat mode that remembers context, one-shot requests, and direct commands that need no LLM at all.
- **Ready for the base shell**: a fixed contract (`AGENT_NAME`, `AGENT_DESCRIPTION`, `handle()`) and a working example base shell that loads, routes to and calls the agent.
- **Evidence**: 129 automated tests, a reproducible demo folder, and a live session in which all requests were answered correctly.

6.2 Challenges Encountered

Managing state and context history across multiple command executions

This was the hardest practical problem. Keeping the whole conversation made every request larger, and on Groq's free tier (8,000 tokens per minute) chat mode hit the rate limit after two or three quick turns. Trimming the history fixed the size but caused a real failure in testing: after a search and two deletions, the follow-up "which files there are hard links of each other?" made the model ask which folder was meant, because the turn that named the folder had been dropped. The solution has three parts:

- keep only the last 6 turns, and tool outputs only for the latest 2;
- keep a small *session note* outside the history (current folder, the last three actions in plain English, and the paths found by the last search) and send it with every request, so "there" and "them" still resolve;
- cap tool results sent to the LLM at 50 items with a count of the rest, while the user's preview always shows everything.

The live session confirms the result: "are there any symlink loops there?" and both "delete them" follow-ups were resolved correctly.

Other challenges

- **The LLM asked for confirmation in text.** Early on, the model previewed a deletion and replied "Would you like me to proceed?", which a one-shot command can never answer. The prompt now tells it to call the write tool directly, and the program asks y/N itself.
- **Provider differences.** Groq's Cloudflare protection rejected Python's default `urllib` User-Agent (HTTP 403, error 1010) until the client sent its own; new Google AI Studio keys did not work with Gemini's OpenAI-compatible endpoint, so Groq became the recommended provider.
- **Loops could not be deleted at first.** The agent found symlink loops but refused to delete them because only "missing target" counted as broken. Loops are now treated as dead links too, with the reason shown in the preview.
- **Tests that depended on the terminal.** Some tests prompted for y/N when run in a real terminal. They now force non-interactive mode and test the prompt with simulated input.

6.3 Future Work

Adding multi-modal support (e.g., voice-controlled terminal commands)

Speech could be turned into text with a local speech-recognition model and passed to the same `handle()` function, so no tool would change. The safety rule must stay strict: a change should still be confirmed explicitly (for example by keyboard, or by reading the preview aloud and asking for a clear "yes"), because speech recognition can mishear paths.

Deepening local file indexing for smarter semantic file search

The agent currently scans on demand. An index of (device, inode) pairs, link targets and link counts, kept up to date with Linux `inotify` events, would answer questions instantly and remove an inherent limit of the current design: hard-link names outside the scanned folder could be found without a full filesystem scan. The same index could support semantic questions such as "which shortcuts point into my old project?".

Further work

- Integrate the agent into the class's base shell once it is built (one entry in its agent list), possibly with an LLM-based router reading every `AGENT_DESCRIPTION`.
- Publish the tools through a Model Context Protocol (MCP) server, which the tool design already matches.
- Instrument end-to-end latency, and test on Python 3.8 and other Linux distributions.

Chapter 7: Screenshots

All screenshots were taken on the student laptop (Ubuntu under WSL) during the session described in Chapter 5. They are shown unedited.

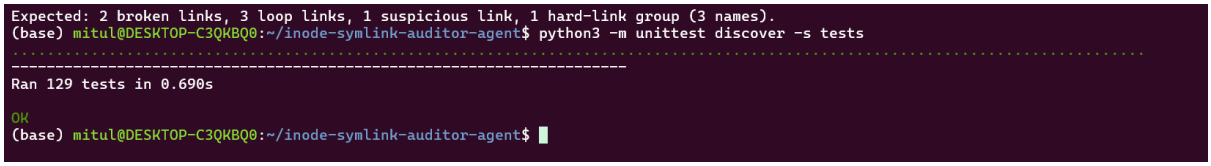


Figure 1: Automated test suite: 129 tests run, OK.

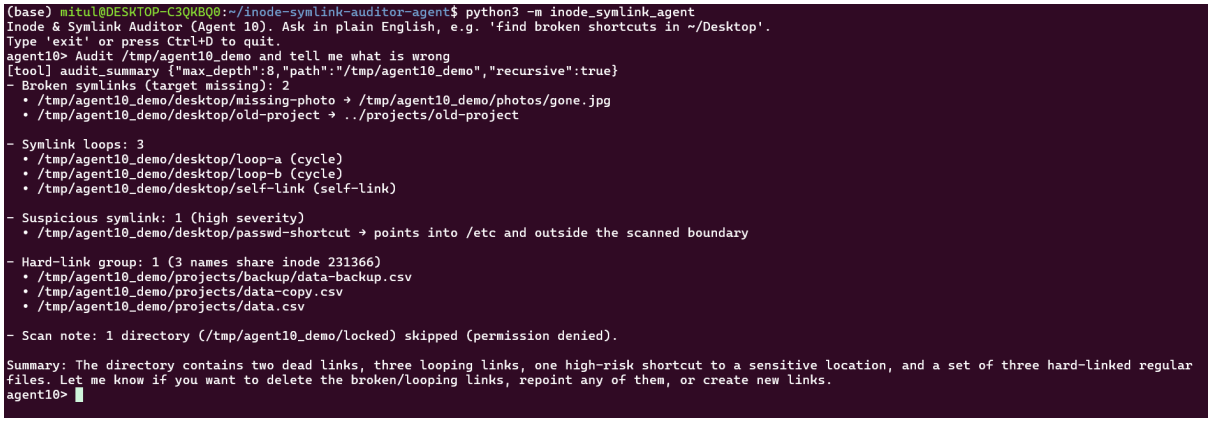


Figure 2: Chat mode: auditing the demo folder. The LLM chose `audit_summary`; every planted problem was found and the unreadable folder was reported as skipped.

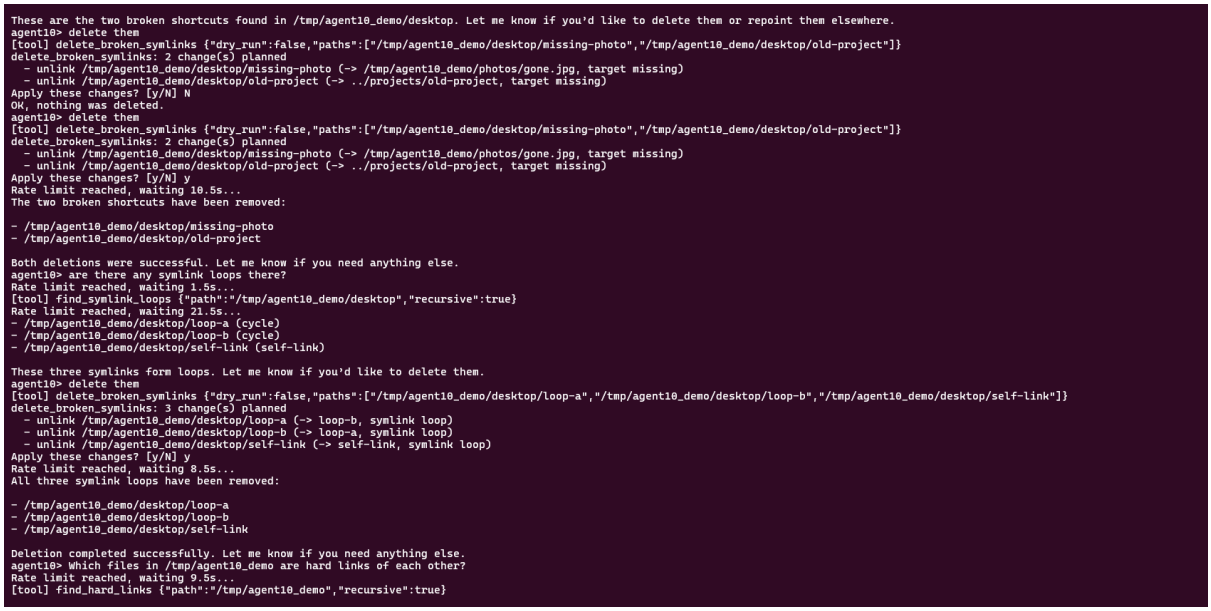


Figure 3: Deleting broken links (first declined with N, then confirmed with y), finding loops "there" from context, and deleting the three loop links. Automatic rate-limit waits are visible.

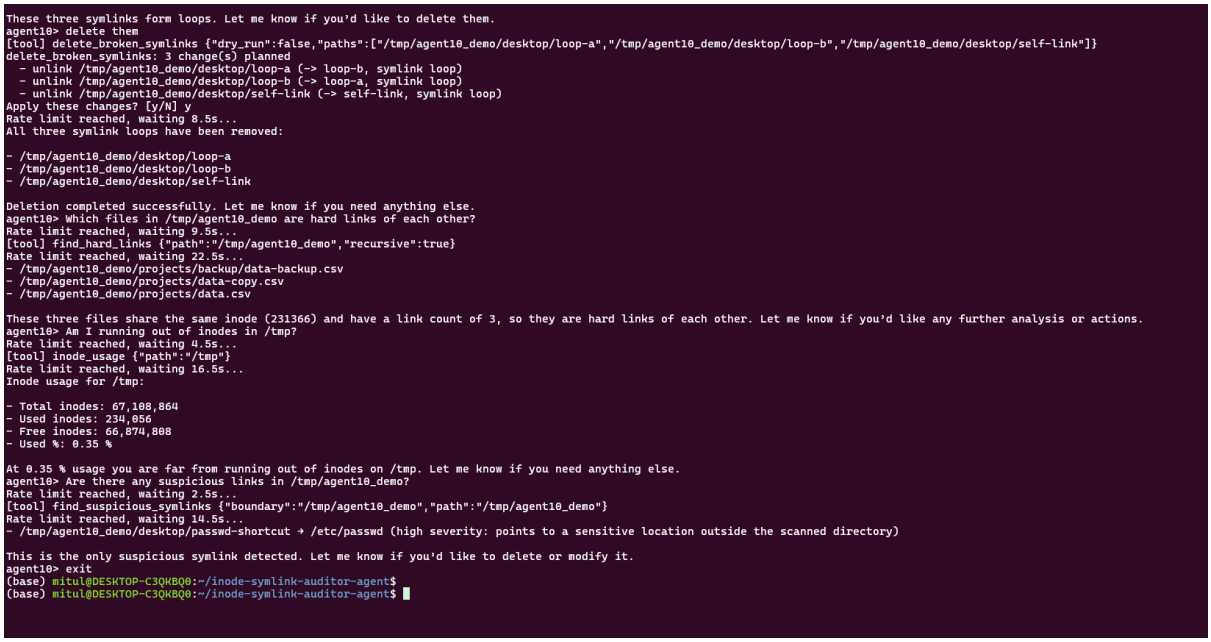


Figure 4: Hard links, inode usage of `/tmp`, and suspicious links.

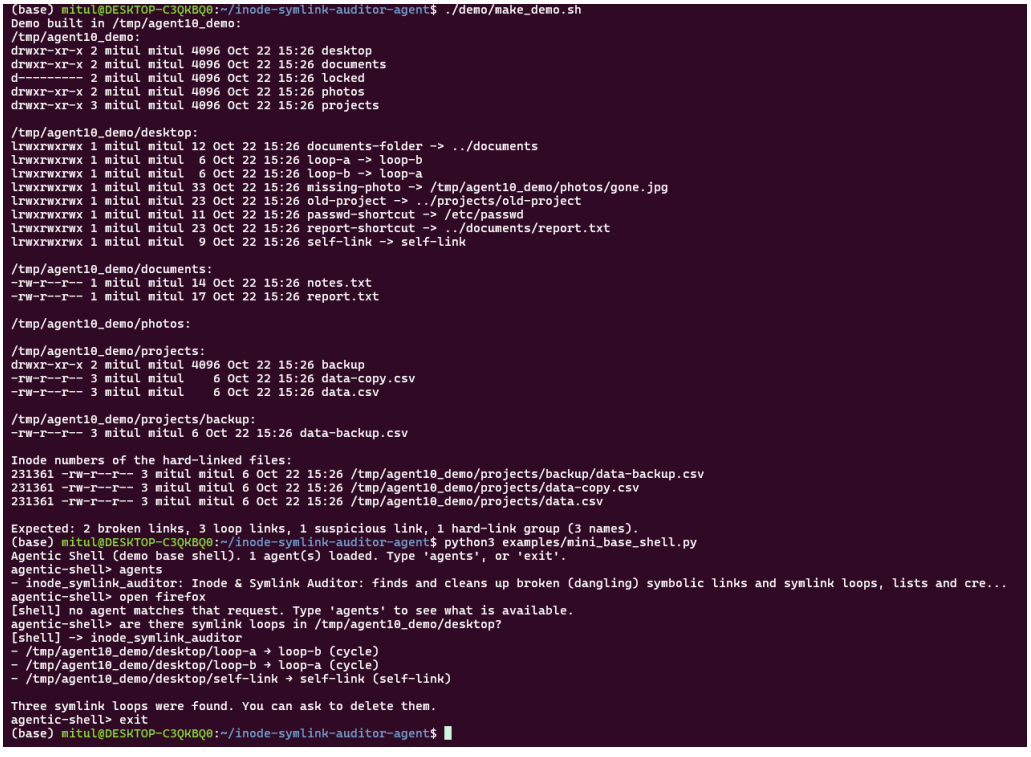


Figure 5: Building the demo folder (with the inode numbers of the hard-linked files), then the example base shell: listing agents, rejecting an unrelated request, and routing a request to this agent.

Project repository: `https://github.com/Debojyoti17/inode-symlink-auditor-agent` (private). The code was developed with AI assistance (Claude), which the course permits.